MyCodeSchool

# React — fișă rapidă

User Management fără server · toate noțiunile din exerciții · Rareș

## 1. Comenzi

| | |
|---|---|
| `npm install` | instalează pachetele (o dată, după clonare) |
| `npm run dev` | pornește aplicația → `http://localhost:5173` · oprire: `Ctrl+C` |
| `npm run build` | verifică dacă proiectul compilează |
| `npm run lint` | caută greșeli (oxlint) |

## 2. Fișierul componentei — ordinea

```
import { useState } from "react";                          ← 1. importuri
type User = { id: number; firstName: string; age: number };  ← 2. tipul datelor

export default function UserList() {                        ← 3. componenta (literă mare)
  const [users, setUsers] = useState<User[]>([...]);       ←    starea
  const canSave: boolean = firstName !== "";                ←    valori calculate din stare
  function remove(id: number): void { ... }                 ←    acțiuni
  return ( <> ... </> );                                    ←    ce se vede (JSX)
}
```

## 3. TypeScript

| | | | |
|---|---|---|---|
| `type User = { id: number; ... }` | forma unui obiect | `User[]` | listă de User |
| `User \| null` | un User sau nimic | `const total: number = 5` | variabilă cu tip |
| `function f(id: number): void` | funcție cu tip | `` `${a} ${b}` `` | text compus |
| `{ id: nextId, firstName }` | `firstName` = `firstName: firstName` | `cond ? a : b` | una din două |
| `a !== b · && · !x` | diferit · și · nu | `Number("30")` | text → număr |

## 4. JSX

| | | | |
|---|---|---|---|
| `{users.length}` | afișează o valoare | `className="card"` | în loc de `class` |
| `onClick={() => remove(user.id)}` | la click, cu argument | `onClick={save}` | la click, fără argument |
| `value={age}` | input controlat | `onChange={(e) => setAge(Number(e.target.value))}` | citește ce scrii |
| `{cond && <p>...</p>}` | doar dacă | `{cond ? <A /> : <B />}` | una din două |
| `{users.map((u) => <tr key={u.id}>...</tr>)}` | listă; key unic | `disabled={!canSave}` | buton inactiv |
| `<br /> · colSpan={3}` | tag-urile se închid; atribute camelCase | `<> ... </>` | mai multe elemente, un singur părinte |

## 5. Starea

| | |
|---|---|
| `const [x, setX] = useState<number>(0)` | valoare + funcția care o schimbă |
| `setX(5)` | valoare nouă → React redesenează componenta |
| `let x = 0; x++` | NU redesenează și se pierde la redesenare (Demo 3) → de aceea `nextId` e stare |
| `users.push(u)` | NU: modifică lista veche, React nu vede → faci o listă nouă |

## 6. Liste și texte

| | | | |
|---|---|---|---|
| `[...users, user]` | listă nouă + un element la final | `users.length` | câte sunt |
| `users.filter((u) => u.id !== id)` | păstrează doar ce trece testul | `user.firstName[0]` | prima literă |
| `users.map((u) => u.id === id ? nou : u)` | înlocuiește un element | `text.toLowerCase()` | litere mici |
| `users.reduce((sum, u) => sum + u.age, 0)` | adună vârstele | `text.includes("pop")` | conține? |
| `Math.round(x * 10) / 10` | o zecimală | | |

## 7. Ce folosești la fiecare exercițiu

| | | | |
|---|---|---|---|
| 1 | `type User` | 6 | `filter`, `onClick={() => ...}` |
| 2 | `import`, `useState<User[]>` | 7 | `value` + `onChange`, `Number()`, `[...users, user]`, `nextId` în stare |
| 3 | `{users.length}` | 8 | `editedId`, `map` pentru înlocuire, `? :` |
| 4 | `map` + `key`, `&&`, `colSpan` | 9 | `const canSave: boolean`, `disabled` |
| 5 | `? :` în `className` | Bonus | `toLowerCase` + `includes`, `reduce`, `User \| null` |